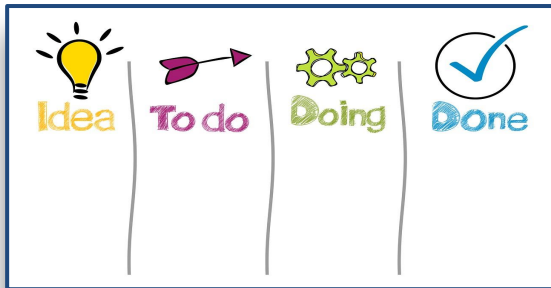


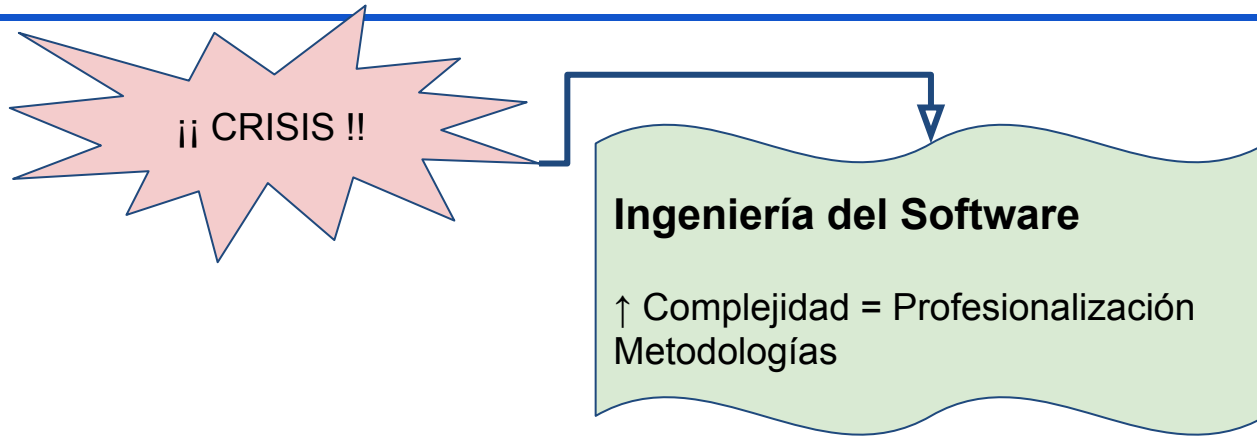
# Tema 1. Introducción al desarrollo ágil.



# Objetivos

- Motivar la necesidad de disponer de metodologías ágiles de desarrollo.
- Comprender por qué las metodologías clásicas o “pesadas” no son adecuadas para **algunos** tipos de proyectos.
- Conocer los valores y principios del manifiesto ágil.

# Crisis del software



*“La Ingeniería del Software es la aplicación de un enfoque **sistemático, disciplinado** y **cuantificable** para el desarrollo, operación y mantenimiento de software: es decir, la aplicación de la ingeniería al software.”*

IEEE Standard Computer Dictionary

## **Cosas que hay que hacer**

### 2 Actividades del desarrollo de software

- 2.1 Planificación
- 2.2 Implementación, pruebas y documentación
- 2.3 Despliegue y mantenimiento

## **Formas de organizarse para hacerlo**

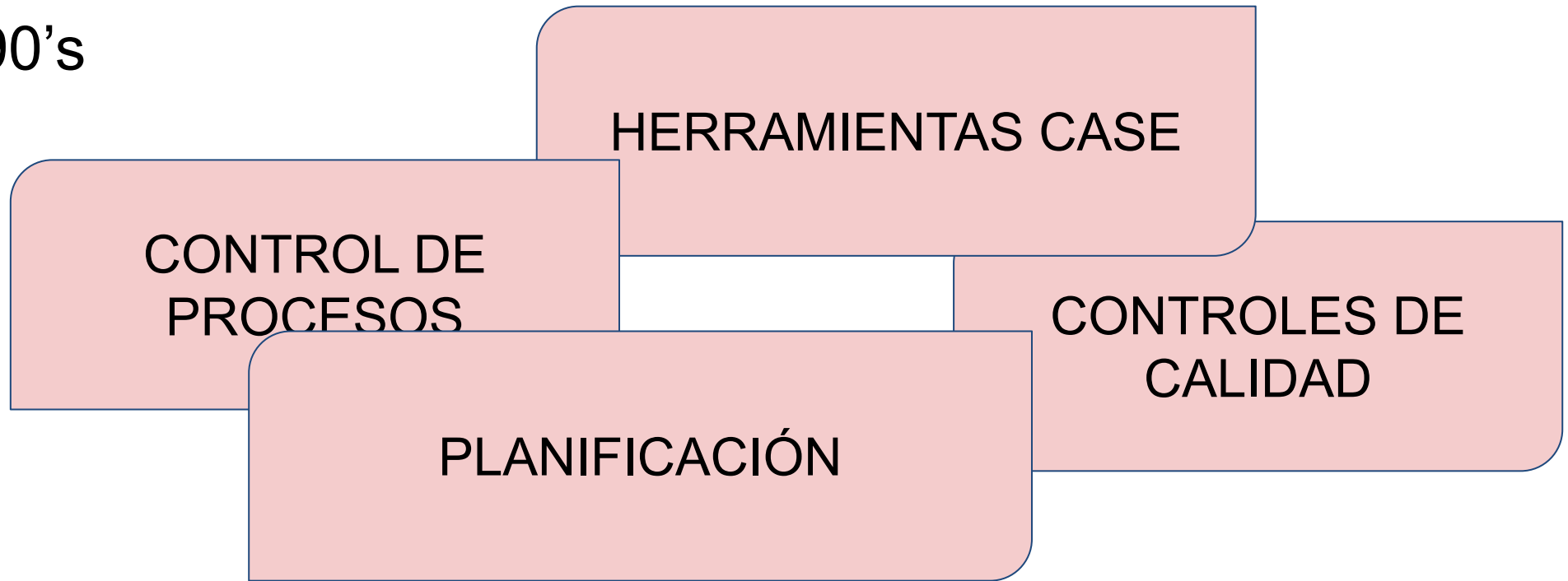
### 3 Modelos de Desarrollo de Software

- 3.1 Modelo de cascada
- 3.2 Modelo de espiral
- 3.3 Desarrollo iterativo e incremental
- 3.4 Desarrollo ágil
- 3.5 Codificación y corrección
- 3.6 Orientado a la Reutilización

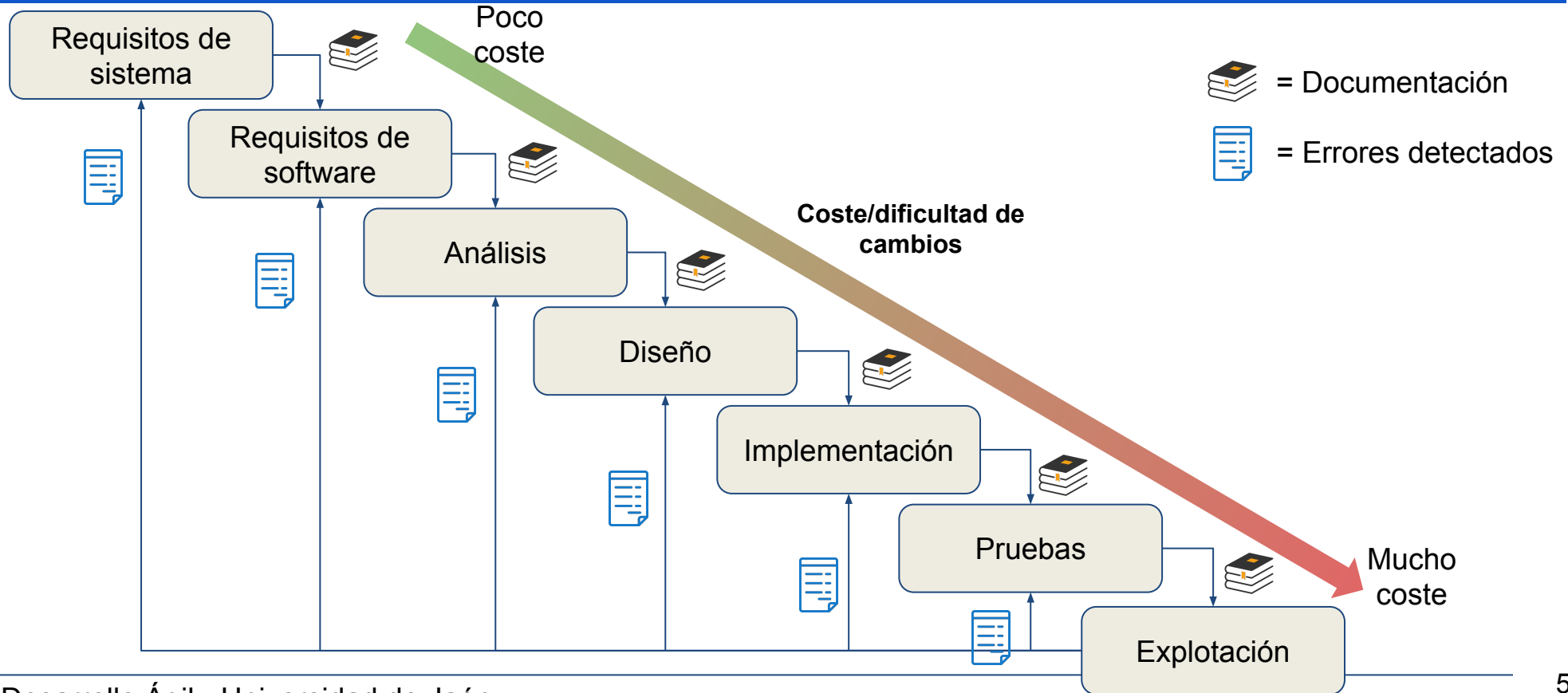
[https://es.wikipedia.org/wiki/Proceso\\_para\\_el\\_desarrollo\\_de\\_software](https://es.wikipedia.org/wiki/Proceso_para_el_desarrollo_de_software)

# Ingeniería de Software “tradicional” (¿pesada?)

<90's



# Modelo de desarrollo en cascada (o Ciclo de Vida Clásico - CVC)



# Una comparación habitual

- El típico ejemplo: desarrollar software es como construir un edificio

- a. Entrevista para saber qué necesita el cliente.
- b. Diseño de los planos.
- c. Cálculos sobre plano.
- d. Revisión, cambios y validación.
- e. Puesta en marcha de la construcción, se permiten cambios menores.
- f. Mantenimiento.



Coste/dificultad de cambios

Poco



Mucho

# Inconvenientes del CVC

RIGIDEZ DEL MODELO  
MUCHO TRABAJO  
MUCHO PAPELEO

PLANIFICACIÓN  
IMPRECISA

REQUISITOS  
CAMBIANTES



CALIDAD  
INADECUADA

PRODUCTIVIDAD !=  
NECESIDADES

**“Men and months are interchangeable commodities only when a task can be partitioned among many workers with no communication among them.”**

The Mythical Man-Month: Essays on Software Engineering.  
Frederick P. Brooks Jr.

*Nueve mujeres no pueden dar a luz 1 hijo en 1 solo mes.*

# Ejemplo de fracaso: *Safeguard*



- *Ballistic Missile Defence System*

- Desarrollado entre 1969 y 1975
- Se emplearon 5407 personas/mes
- El hardware se diseñó a la vez que se escribían las especificaciones del software
- No se permitieron cambios en los requerimientos

- ¿Cómo se desarrolló?

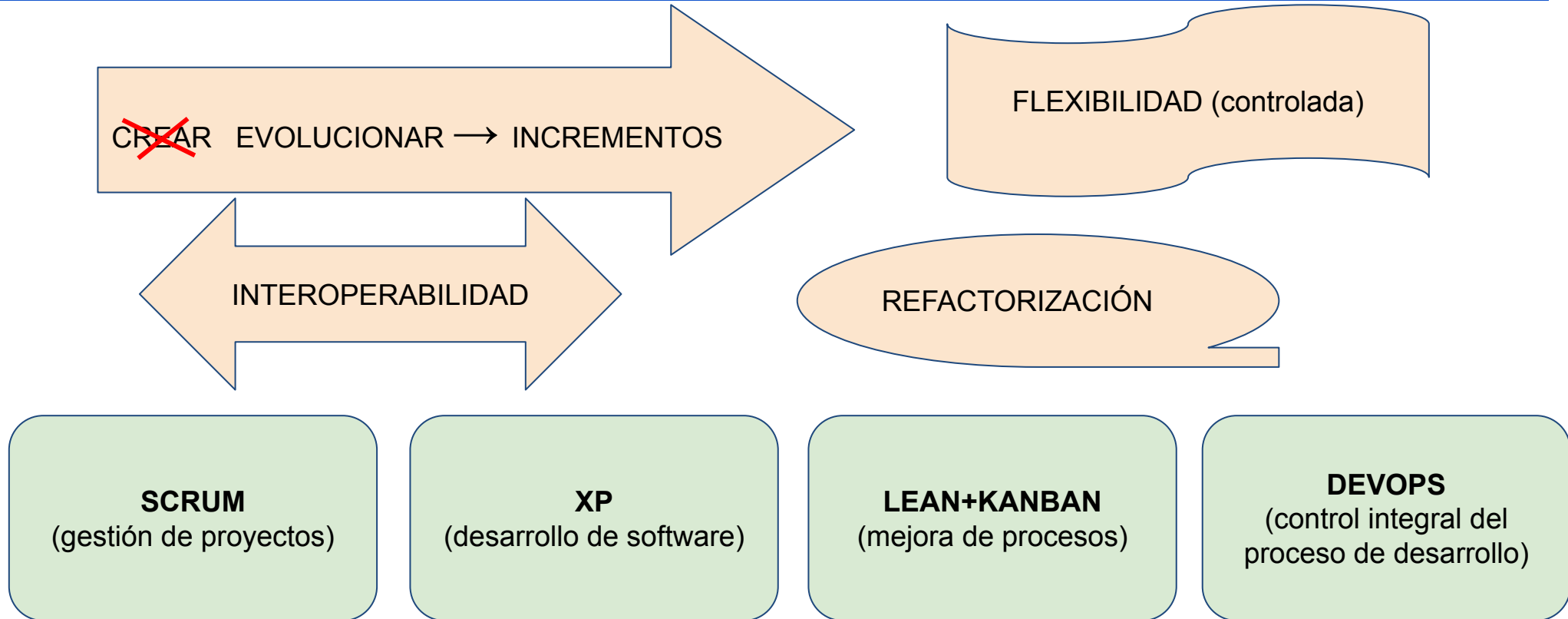
- El proyecto se desarrolló de acuerdo con las especificaciones
- Coste (sin ajustar): \$25.000 Millones
- 6 años de desarrollo (1969-1975), 5407 personas/mes
- El proyecto estuvo operativo 133 días, y fue cancelado en 1978
- **Cuando el sistema anti-misiles se terminó a los 6 años... los nuevos misiles eran más rápidos que el sistema anti-misiles**

- ¿Qué se hizo mal?

- La metodología de Ingeniería del Software utilizada era “pesada”, y este tipo de metodologías tienen más éxito cuando:
  - Los requerimientos son ESTABLES
  - Todo ocurre como se espera
  - No manejamos cosas nuevas o desconocidas
  - Se han desarrollado muchos proyectos similares antes
- El problema es que **en la vida real hay muy pocos proyectos que tengan estas características**



# Nuevos paradigmas



# El manifiesto ágil

2001,  
*Agile Alliance*

## Manifesto for Agile Software Development

4

valores

12

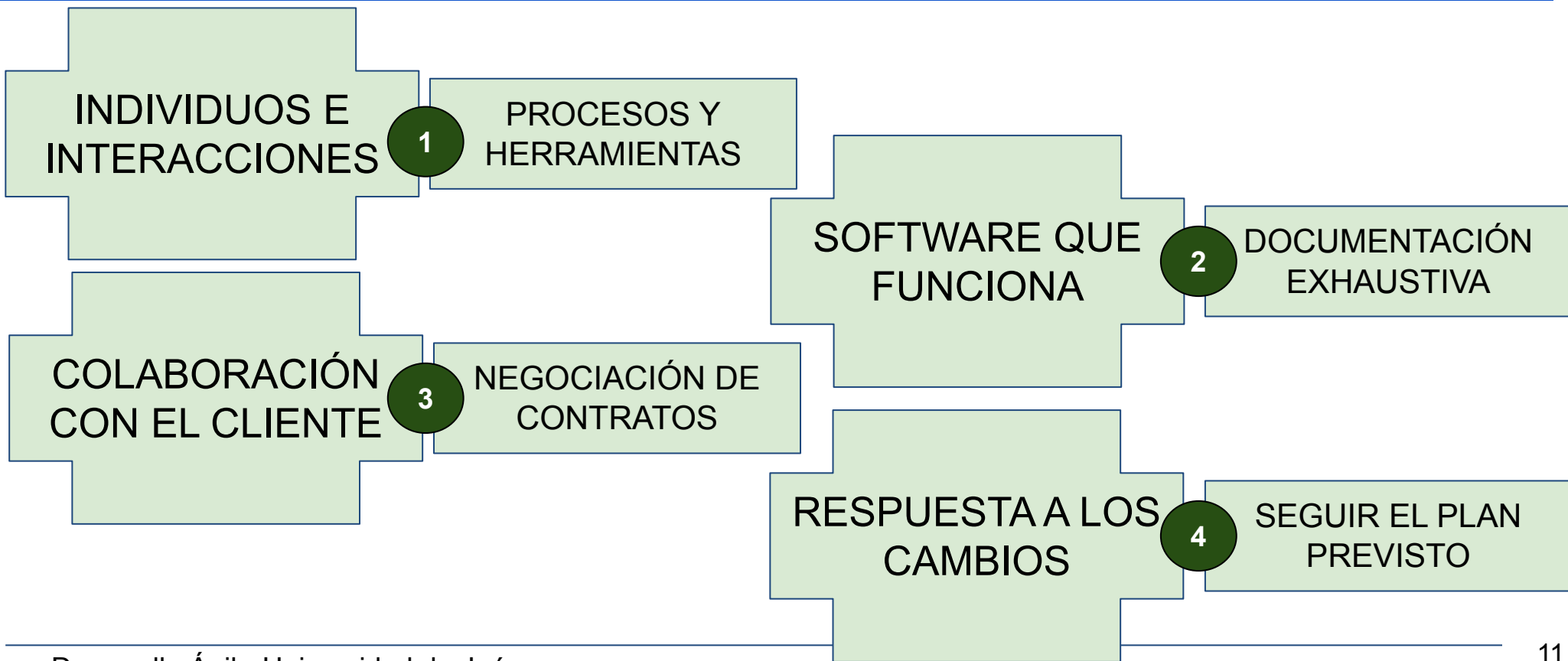
principios

<https://agilemanifesto.org>

(<https://agilemanifesto.org/iso/es/manifesto.html>) 

XP (*eXtreme Programming*), Scrum, DSDM (*Dynamic Systems Development Method*), Adaptive Software Development, Crystal, Feature-Driven Development, Pragmatic Programming, ...

# El manifiesto ágil: valores



# El manifiesto ágil: principios 1 a 4

1. **Nuestra mayor prioridad** es satisfacer al cliente mediante la **entrega temprana y continua de software con valor**.
2. **Aceptamos que los requisitos cambien**, incluso en etapas tardías del desarrollo. Los procesos Ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. **Entregamos software funcional frecuentemente**, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto posible.
4. Los responsables de negocio y los desarrolladores **trabajamos juntos de forma cotidiana** durante todo el proyecto.

# El manifiesto ágil: principios 5 a 8

5. Los proyectos se desarrollan en torno a **individuos motivados**. Hay que darles el entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.
6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la **conversación cara a cara**.
7. El software funcionando es la **medida principal de progreso**.
8. Los procesos ágiles promueven el **desarrollo sostenible**. Los promotores, desarrolladores y usuarios debemos ser capaces de mantener un **ritmo constante** de forma indefinida.

# El manifiesto ágil: principios 9 a 12

9. La atención continua a la **excelencia técnica y al buen diseño** mejora la Agilidad.
10. **La simplicidad**, o el arte de maximizar la cantidad de trabajo no realizado, **es esencial**.
11. Las mejores arquitecturas, requisitos y diseños emergen de **equipos auto-organizados**.
12. A intervalos regulares el equipo reflexiona sobre **cómo ser más efectivo** para a continuación **ajustar y perfeccionar su comportamiento** en consecuencia.

# Situación actual del uso de tecnologías ágiles

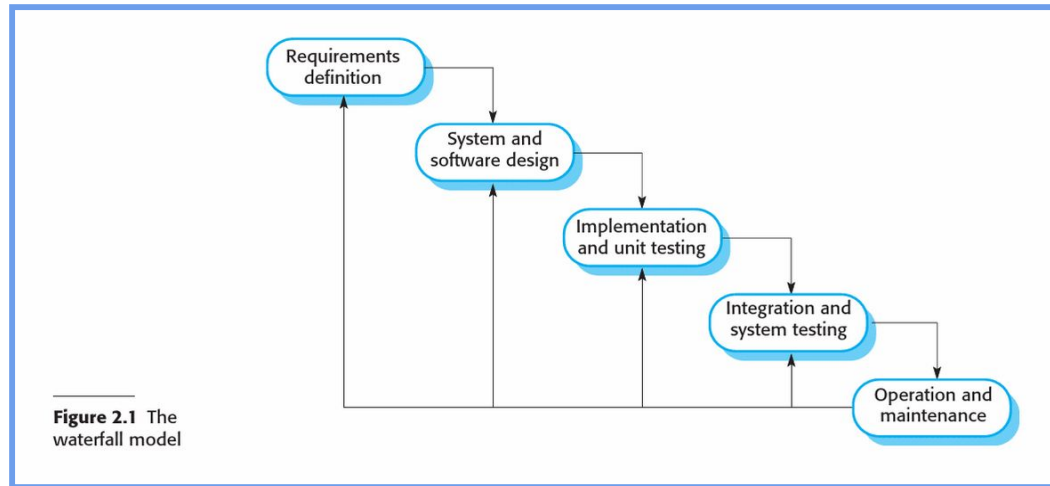
- Analizar y extraer conclusiones del documento:

## *17th annual State of Agile Report*

- <https://drive.google.com/file/d/1JaKJruuxlvkKdRPq6yNVnCZ2ra4OrcFE/view>

# Comparación de metodologías de desarrollo

## Metodología en Cascada o CVC



1. Planifico todo
2. Análizo todo
3. Diseño todo
4. Implemento y pruebo todo
5. Integro y valido todo
6. Despliegue todo

Ian Sommerville, (2011). INGENIERÍA DEL SOFTWARE 9ED. 9. [ebook] Madrid. España: Pearson.  
Disponible en: [https://www.ingebook.com/ib/NPcd/IB\\_BooksVis?cod\\_primaria=1000193&codigo\\_libro=1518](https://www.ingebook.com/ib/NPcd/IB_BooksVis?cod_primaria=1000193&codigo_libro=1518)  
[Accedido el 2024-07-25 20:12:07].



# Comparación de metodologías de desarrollo

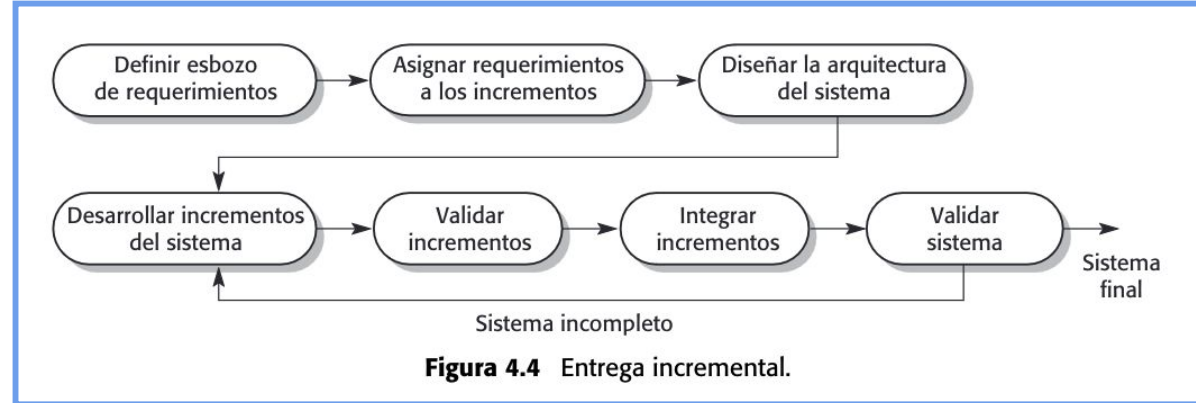
## Metodología en Cascada o CVC

- **Objetivo:** minimizar los errores cometidos en cada fase para no propagarlos de forma aumentada en las siguientes.
- **Útil** cuando conozco perfectamente las funcionalidades a desarrollar y tengo claro que lo voy a implementar todo.
- **Problemas** si necesito definir nuevas funcionalidades en tiempo de desarrollo

# Comparación de metodologías de desarrollo

## Metodología Incremental

1. Planifico todo
2. Analizo todo
3. Diseño todo (o casi) todo.
4. Llevo a cabo el primer incremento:
  - a. Selecciono algunas funcionalidades
  - b. Implemento dichas funcionalidades en un incremento
  - c. Despliego el resultado
5. Continúo con el resto de funcionalidades creando incrementos.



**Figura 4.4** Entrega incremental.

Ian Sommerville, (2011). INGENIERÍA DEL SOFTWARE 9ED. 9. [ebook] Madrid. España: Pearson.  
Disponible en: [https://www.ingebook.com/ib/NPcd/IB\\_BooksVis?cod\\_primaria=1000193&codigo\\_libro=1518](https://www.ingebook.com/ib/NPcd/IB_BooksVis?cod_primaria=1000193&codigo_libro=1518) [Accedido el 2024-07-25 20:12:07].

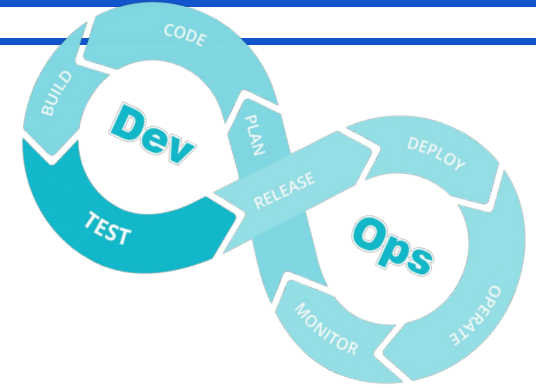
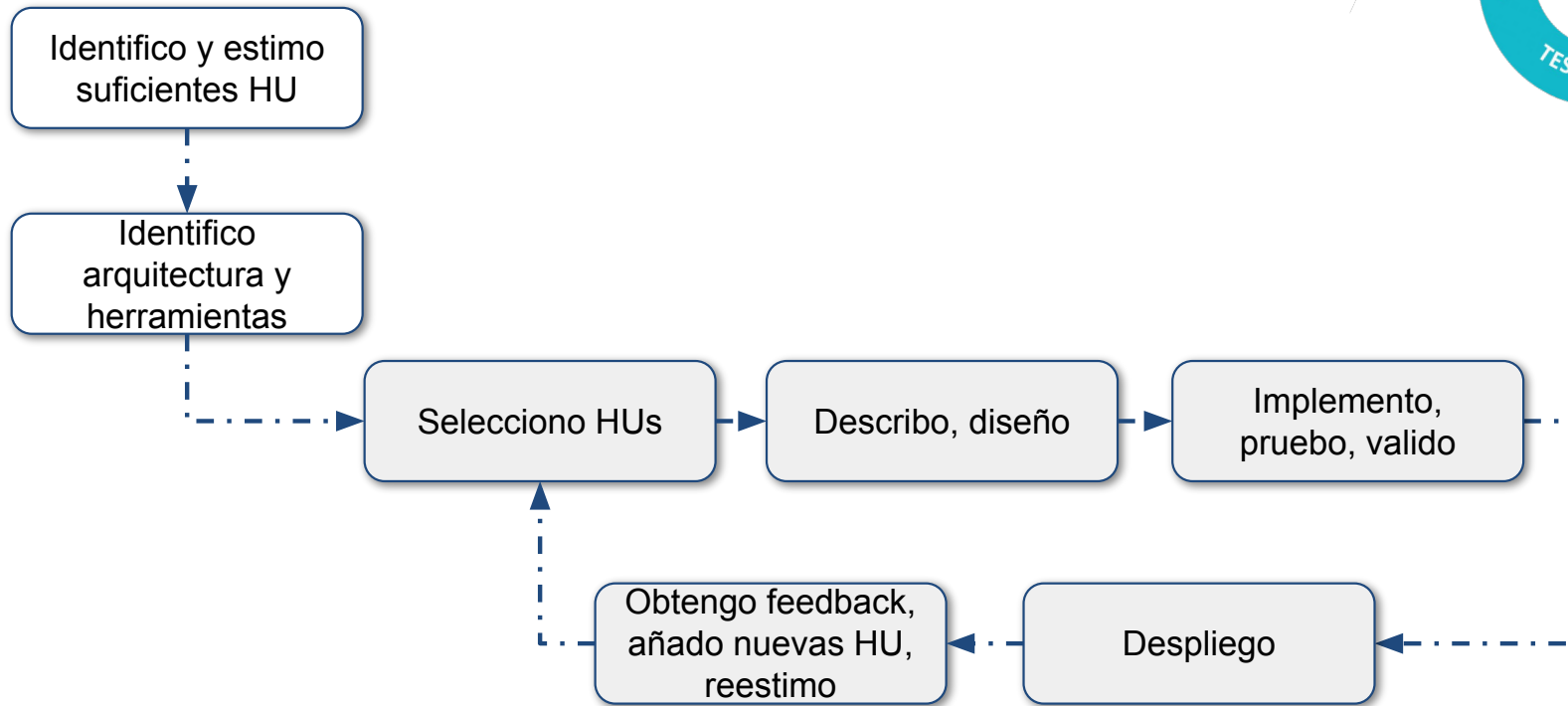
# Comparación de metodologías de desarrollo

## Metodología Incremental

- **Objetivo:** utilizar la filosofía del CVC pero obteniendo versiones ejecutables antes de tener todo implementado.
- **Útil** cuando tenemos una idea clara de qué queremos implementar, pero queremos tener al menos una aplicación funcional que poder mostrar a los usuarios (=tribunal) o presentar el MVP sin necesidad de llegar a tenerlo todo terminado.
- **Problemas** para añadir nuevas funcionalidades detectadas tras empezar a entregar los incrementos.

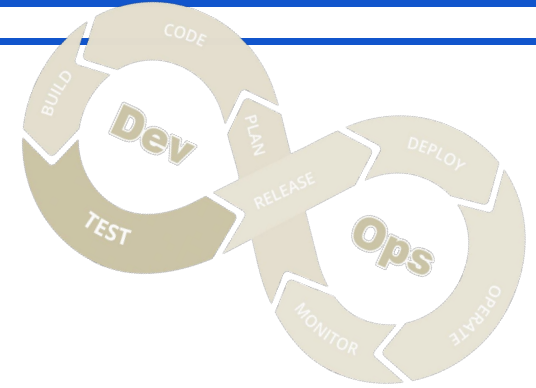
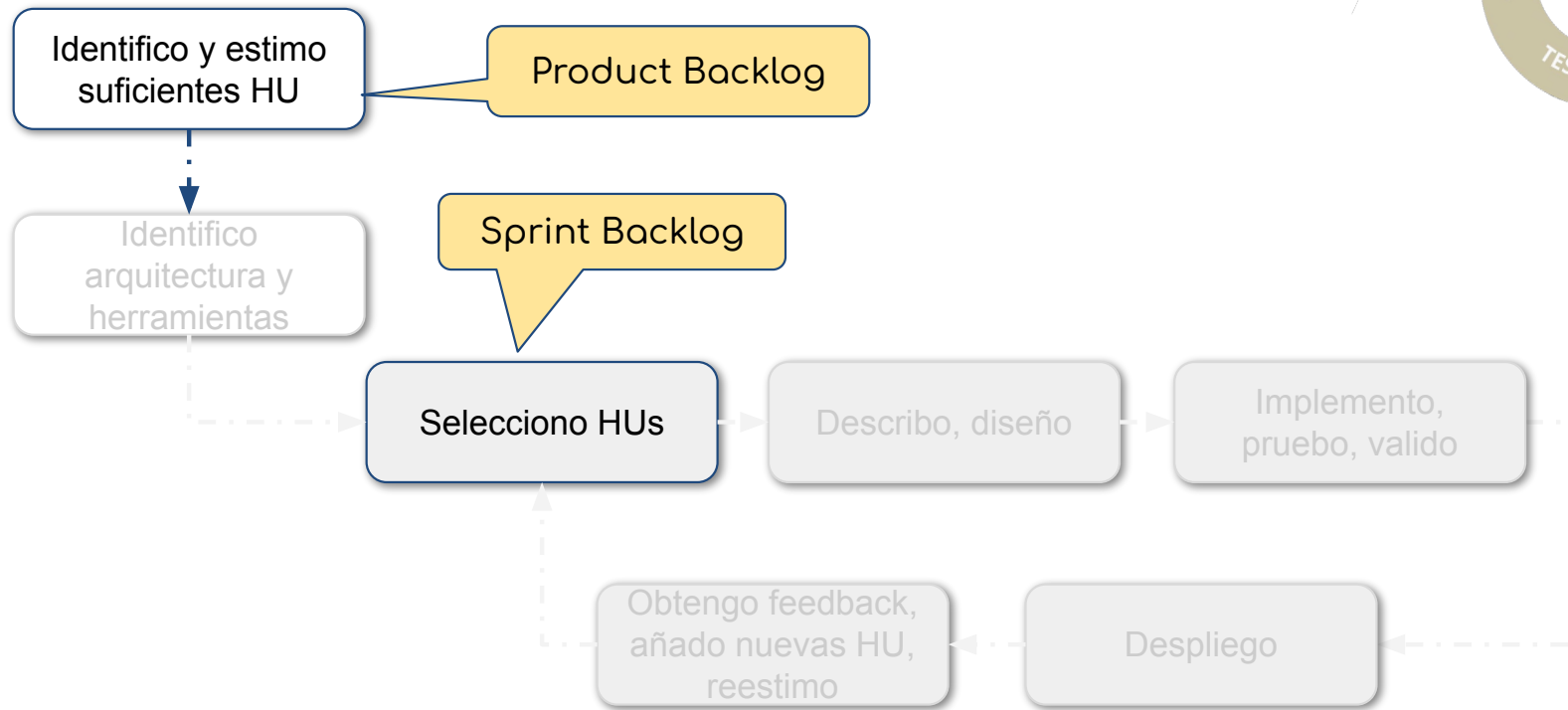
# Comparación de metodologías de desarrollo

## Metodologías ágiles



# Comparación de metodologías de desarrollo

## Metodologías ágiles



# Comparación de metodologías de desarrollo

## Metodologías ágiles

- **Objetivo:** maximizar la cantidad de trabajo que **NO** hay que llevar a cabo.
- **Útil** cuando no es fácil determinar cuánto será el tiempo que nos costará implementar las funcionalidades, así como cuando no tenemos una idea exacta de hacia dónde evolucionará nuestro software pues éste depende mucho del feedback de los usuarios.
- **Problemas** para seleccionar correctamente las HU de cada incremento y obtener feedback significativo de los usuarios.

# Terminología básica en metodologías ágiles

- HU: historias de usuario

- Es una **descripción escrita** de parte de la **funcionalidad** del software, expresada de forma **valiosa** para las partes involucradas y **útil** para la gestión de proyectos.

*Como <tipo de usuario>  
quiero <acción concreta>  
para <lo que quiero que  
ocurra como resultado>*

- *done* (“hecho”)

- Estado en el que se puede pasar una HU una vez que ha sido implementada, probada y validada por los usuarios. **Difícil de establecer.**

- Cultura de una empresa

- Conjunto compartido de valores, metas, actitudes y prácticas (o metodologías) que definen a cada organización.

# Terminología básica en metodologías ágiles (II)

- *Workflow*
  - Conjunto de fases que deben ser realizadas para completar un proceso **repetitivo**.
- Application Life Management (ALM)
  - personas+herramientas+procesos
- CI/CD: *Continuous Integration/Continuous Delivery-Deployment*